

POA Projectplan: Web Security Toolkit

Student: [Jouw naam] **Opleiding:** Informatica – jaar 2 **Periode:** Mei 2026 – Juni 2026 **Totale uren:** 28 uur

1. Inleiding

Dit document beschrijft het gedetailleerde uitvoeringsplan voor de Persoonlijke Ontwikkel Activiteit (POA). Het doel is het bouwen van een webapplicatie genaamd **Web Security Toolkit**, bestaande uit twee uitgebreide modules:

1. **Web Security Scanner** – analyseert een URL op security headers, open poorten, kwetsbaarheden en genereert een rapport.
 2. **Password Strength Analyzer** – beoordeelt wachtwoorden op sterkte, berekent entropie, detecteert patronen en controleert op bekende datalekken.
-

2. Leerdoelen

- Zelfstandig een full-stack webapplicatie opzetten en afronden binnen 28 uur.
 - Werken met React (frontend) en een gesplitste backend: Node.js voor API-routing en Python voor security-analyse.
 - Diepgaande kennis opdoen van cybersecurity concepten: HTTP security headers, poortscannen, wachtwoordentropie en credential leaks.
 - Ervaring opdoen met externe API-integraties (Have I Been Pwned).
 - Een project publiek delen via GitHub inclusief volledige documentatie.
-

3. Stack-keuze & motivatie

Waarom een gesplitste backend (Node.js + Python)?

De backend is opgesplitst in twee verantwoordelijkheden:

- **Node.js (Express)** fungeert als de centrale API-gateway. Het ontvangt alle verzoeken van de frontend, handelt routing af en stuurt waar nodig door naar de Python-service. Node.js is hier ideaal vanwege zijn snelheid bij I/O-taken (HTTP-verzoeken ophalen, headers analyseren).
- **Python (FastAPI)** neemt de zware security-analyse voor zijn rekening: poortscannen via `python-nmap`, wachtwoordentropie berekenen en patroondetectie. Python heeft hiervoor een veel rijker ecosysteem aan security-bibliotheken dan Node.js.

Deze aanpak is ook realistischer voor de industrie: microservices met aparte verantwoordelijkheden per taal.

4. Tech Stack

Frontend

Technologie	Versie	Reden
React	18.x	Component-based UI, breed gebruikt in de industrie
Vite	5.x	Snelle development server en build tool
Axios	1.x	HTTP-verzoeken naar de backend
TailwindCSS	3.x	Snelle, utility-first styling
Recharts	2.x	Visualisatie van rapport/score (grafieken)

Backend – Node.js (API Gateway)

Technologie	Versie	Reden
Node.js	20.x LTS	JavaScript runtime, snel bij I/O
Express	4.x	Minimalistisch web framework
Axios	1.x	Headers ophalen van externe websites
cors	2.x	Cross-Origin verzoeken toestaan
dotenv	16.x	Omgevingsvariabelen beheren
crypto (built-in)	–	SHA-1 hash genereren voor HIBP k-anonimiteit

Backend – Python (Security Engine)

Technologie	Versie	Reden
Python	3.12	Rijk security-ecosysteem
FastAPI	0.11x	Modern, snel async web framework
uvicorn	0.29	ASGI server voor FastAPI
python-nmap	0.7.x	Poortscannen via nmap
requests	2.x	HTTP-verzoeken vanuit Python
math (built-in)	–	Entropieberekening

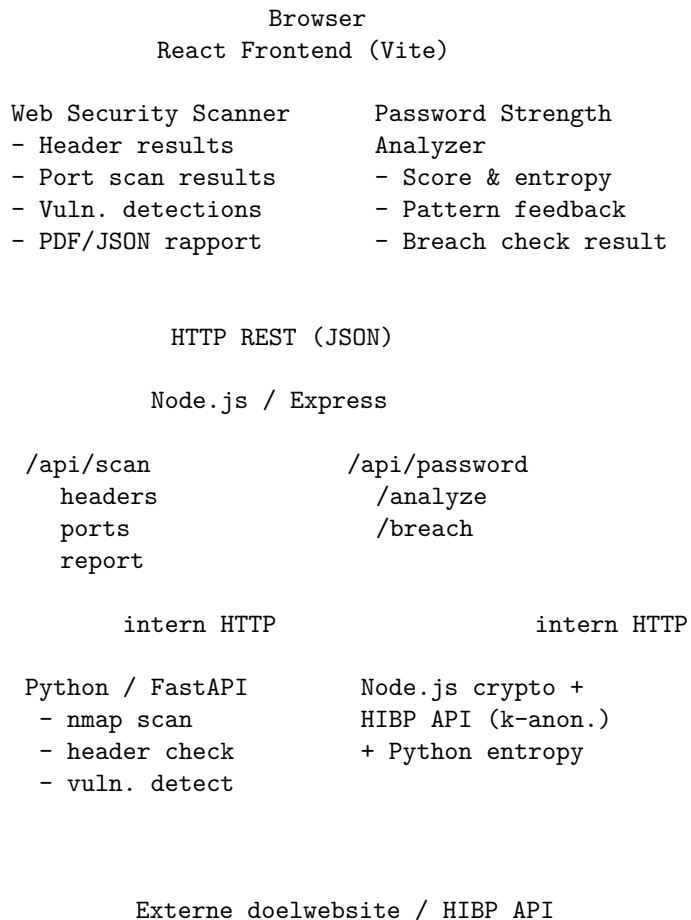
Externe API's

API	Doel
Have I Been Pwned (HIBP) Passwords API	Controleren of wachtwoord voorkomt in bekende datalekken

Tooling

Tool	Doel
Git & GitHub	Versiebeheer en publicatie
VS Code	Code-editor
Postman	API testen
nmap	Netwerk-/poortscanner (systeemtool, vereist installatie)
Docker (optioneel)	Omgeving consistent houden over machines

5. Architectuur



6. Functionaliteiten

6.1 Web Security Scanner

De gebruiker voert een URL in en krijgt een volledig rapport terug.

6.1.1 Security Header Check De backend haalt de HTTP-response headers op van de opgegeven URL en controleert:

Header	Beveiligingsdoel
Content-Security-Policy	Voorkomt XSS-aanvallen
Strict-Transport-Security	Dwingt HTTPS af (HSTS)
X-Content-Type-Options	Voorkomt MIME-sniffing
X-Frame-Options	Beschermt tegen clickjacking
Referrer-Policy	Beperkt referrer-informatie
Permissions-Policy	Beheert browser-API-toegang
X-XSS-Protection	Legacy XSS-filter (browsers)
Cache-Control	Voorkomt onveilige caching van gevoelige data

Per header: aanwezig / ontbreekt , de feitelijke waarde en een uitleg.

6.1.2 Basis Poortscan Via `python-nmap` worden de meest gangbare poorten gescand van het hostname van de URL:

Poort	Service	Risico als open
21	FTP	Onversleutelde bestandsoverdracht
22	SSH	Brute-force doelwit
23	Telnet	Volledig onversleuteld
25	SMTP	Mailserver, spam-risico
80	HTTP	Onversleuteld webverkeer
443	HTTPS	Normaal open, SSL checken
3306	MySQL	Database blootgesteld
5432	PostgreSQL	Database blootgesteld
8080	HTTP-alt	Dev-server publiek?
27017	MongoDB	Database blootgesteld

Ethische noot: De poortscan wordt alleen uitgevoerd op de host van de ingevoerde URL. Het is de verantwoordelijkheid van de gebruiker om alleen websites te scannen waarvoor zij toestemming hebben.

6.1.3 Kwetsbaarheidsdetectie (simpel) Op basis van de gecombineerde resultaten worden simpele kwetsbaarheden gesignaleerd:

- HTTP gebruikt in plaats van HTTPS → “Onversleuteld verkeer”
- **Strict-Transport-Security** ontbreekt terwijl HTTPS actief is → “HSTS niet geconfigureerd”
- Database-poort (3306, 5432, 27017) open → “Database mogelijk publiek bereikbaar”
- **Content-Security-Policy** ontbreekt → “XSS-risico aanwezig”
- **X-Frame-Options** ontbreekt → “Clickjacking mogelijk”
- Telnet (23) open → “Onveilig protocol actief”

6.1.4 Rapport Na de scan wordt een overzichtelijk rapport samengesteld met:

- Totaalscore (bijv. 6/10)
- Samenvatting van gevonden issues per categorie
- Aanbevelingen per gevonden kwetsbaarheid
- Export als JSON (en optioneel als PDF via **jsPDF** in de frontend)

6.2 Password Strength Analyzer

6.2.1 Sterkte-analyse

- **Score 0–4** op basis van zxcvbn (bewezen bibliotheek, getraind op echte aanvalspatronen).
- **Visuele sterkte-balk** (rood → oranje → geel → groen).
- **Geschatte kraaktijd** (offline en online aanval).

6.2.2 Entropieberekening De entropie wordt berekend via: $H = L \times \log(N)$

waarbij L de wachtwoordlengte is en N de karakterset-grootte (bijv. 26 voor alleen kleine letters, 95 voor alle printbare ASCII-tekenen). De Python-backend voert deze berekening uit en retourneert de waarde in bits.

Entropie	Beoordeling
< 28 bits	Zeer zwak
28–35 bits	Zwak
36–59 bits	Redelijk
60–127 bits	Sterk
128 bits	Zeer sterk

6.2.3 Patroondetectie De backend detecteert veelvoorkomende onveilige patronen:

- Toetsenbord-reeksen: **qwerty**, **123456**, **asdfgh**
- Herhalingen: **aaaa**, **1111**
- Veelgebruikte woorden: **password**, **welkom**, **admin**

- Datum-patronen: 1990, 2024, ddmmjjjj
- Leetspeak-varianten: p@ssw0rd (wordt herkend als zwak patroon)

6.2.4 Breached Password Check (Have I Been Pwned) Integreert met de **HIBP Passwords API** via het **k-anonimiteit model** – het volledige wachtwoord wordt *nooit* verstuurd:

1. Frontend/Node.js berekent een SHA-1 hash van het wachtwoord.
2. Alleen de **eerste 5 karakters** van de hash worden naar de HIBP API gestuurd.
3. HIBP retourneert alle hashes die met die 5 karakters beginnen.
4. Lokaal wordt gecheckt of de volledige hash erbij zit.
5. Resultaat: “Wachtwoord gevonden in X datalekken” of “Niet gevonden”.

7. API Endpoints

Node.js (poort 3000)

POST /api/scan/headers

```
// Request
{ "url": "https://example.com" }

// Response
{
  "url": "https://example.com",
  "headers": {
    "Content-Security-Policy": { "present": true, "value": "default-src 'self'", "risk": "low" },
    "Strict-Transport-Security": { "present": false, "value": null, "risk": "high" }
  },
  "headerScore": 5,
  "maxHeaderScore": 8
}
```

POST /api/scan/ports

```
// Request
{ "url": "https://example.com" }

// Response
{
  "host": "example.com",
  "ports": [
    { "port": 22, "state": "open", "service": "ssh", "risk": "medium" },
    { "port": 3306, "state": "closed", "service": "mysql", "risk": "high" }
  ]
}
```

```

    ]
}

POST /api/scan/report

// Request
{ "url": "https://example.com" }

// Response
{
  "url": "https://example.com",
  "totalScore": 6,
  "maxScore": 10,
  "grade": "C",
  "vulnerabilities": [
    { "id": "MISSING_HSTS", "severity": "high", "title": "HSTS ontbreekt", "recommendation": "..." },
  ],
  "headers": { "...": "..." },
  "ports": [ "..." ],
  "generatedAt": "2026-05-10T14:00:00Z"
}

POST /api/password/analyze

// Request
{ "password": "myP@sswOrd" }

// Response
{
  "score": 2,
  "label": "Zwak",
  "entropy": 34.7,
  "entropyLabel": "Zwak",
  "crackTimeOnline": "3 uur",
  "crackTimeOffline": "2 seconden",
  "patterns": ["leetspeak", "common-word"],
  "feedback": {
    "warning": "Lijkt op een veelgebruikt wachtwoord",
    "suggestions": ["Gebruik een langere zin", "Voeg speciale tekens toe"]
  }
}

POST /api/password/breach

// Request
{ "hashPrefix": "5BAA6" }

```

```
// Response
{
  "breached": true,
  "occurrences": 34201,
  "message": "Dit wachtwoord is 34.201 keer gevonden in bekende datalekken."
}
```

Python / FastAPI (poort 8000, intern)

Endpoint	Methode	Functie
/scan/headers	POST	Header-analyse
/scan/ports	POST	Nmap poortscan
/scan/vulnerabilities	POST	Kwetsbaarheidsdetectie
/password/entropy	POST	Entropieberekening
/password/patterns	POST	Patroondetectie

8. Mappenstructuur

web-security-toolkit/

```
client/                                # React frontend
  src/
    components/
      scanner/
        ScannerForm.jsx               # URL-invoer
        HeaderResults.jsx             # Header-tabel
        PortResults.jsx               # Poort-overzicht
        VulnList.jsx                  # Kwetsbaarheden
        ReportCard.jsx                # Eindrapport + export
      password/
        PasswordForm.jsx              # Wachtwoordinvoer
        StrengthBar.jsx               # Visuele balk
        EntropyGauge.jsx              # Entropie-indicator
        PatternWarnings.jsx           # Patroonwaarschuwingen
        BreachResult.jsx              # HIBP resultaat
      shared/
        ScoreCircle.jsx               # Cirkelscore
        Badge.jsx                     # Status-badges
    pages/
      ScannerPage.jsx
      PasswordPage.jsx
    services/
      api.js                           # Axios API-calls
```



```

    App.jsx
    main.jsx
    index.css
index.html
vite.config.js
package.json

server-node/                                # Node.js API Gateway
  routes/
    scan.js                                # /api/scan/*
    password.js                            # /api/password/*
  services/
    headerService.js                      # Headers ophalen & checken
    hibpService.js                        # HIBP API aanroepen
    pythonBridge.js                       # Intern HTTP naar FastAPI
  middleware/
    validate.js                           # Input-validatie
  index.js
  .env
  package.json

server-python/                              # Python Security Engine
  routers/
    scan.py                               # /scan/* endpoints
    password.py                           # /password/* endpoints
  services/
    header_analyzer.py                   # Header-analyse logica
    port_scanner.py                     # nmap wrapper
    vuln_detector.py                    # Kwetsbaarheidslogica
    entropy_calculator.py               # Entropie & patronen
  main.py
  requirements.txt

.gitignore
README.md

```

9. Tijdsplanning (28 uur)

Fase 1 – Voorbereiding & opzet (3,5 uur)

Taak	Uren
Git repo aanmaken, mapstructuur opzetten	0,5
Node.js backend opzetten (Express + basis routing)	0,75

Taak	Uren
Python backend opzetten (FastAPI + uvicorn)	0,75
React frontend opzetten (Vite + Tailwind)	0,75
Nmap installeren en testen, HIBP API verkennen	0,75

Fase 2 – Web Security Scanner backend (8 uur)

Taak	Uren
<code>headerService.js</code> : headers ophalen en checken (8 headers)	1,5
<code>header_analyzer.py</code> : risico-beoordeling per header	1
<code>port_scanner.py</code> : nmap wrapper voor 10 poorten	2
<code>vuln_detector.py</code> : kwetsbaarheidsdetectie op basis van resultaten	1,5
Rapport-endpoint samenstellen + scoring systeem	1,5
Testen via Postman	0,5

Fase 3 – Password Analyzer backend (4 uur)

Taak	Uren
<code>entropy_calculator.py</code> : entropieformule implementeren	1
Patroondetectie (reeksen, herhalingen, veelgebruikte woorden)	1
<code>hibpService.js</code> : SHA-1 hash + k-anonimiteit HIBP call	1,5
Testen met diverse wachtwoorden	0,5

Fase 4 – Frontend ontwikkeling (8 uur)

Taak	Uren
Navigatiestructuur en paginalayout	0,5
Scanner: <code>ScannerForm</code> , <code>HeaderResults</code> , <code>PortResults</code>	2,5
Scanner: <code>VulnList</code> , <code>ReportCard</code> + JSON-export	1,5
Password: <code>PasswordForm</code> , <code>StrengthBar</code> , <code>EntropyGauge</code>	1,5
Password: <code>PatternWarnings</code> , <code>BreachResult</code>	1
Globale foutafhandeling en laadstates	0,5

Fase 5 – Testen & afwerking (2,5 uur)

Taak	Uren
End-to-end testen (meerdere URLs, wachtwoorden)	1
Bugfixes en code opschonen	0,75
README voltooien met installatie-instructies + screenshots	0,75

Fase 6 – Reflectie & documentatie (2 uur)

Taak	Uren
Reflectieverslag schrijven (wat geleerd, wat ging mis)	1,5
Screenshots en demo-materiaal verzamelen, GitHub pushen	0,5

Totaal: 28 uur

10. Risico's & mitigatie

Risico	Kans	Impact	Aanpak
Nmap werkt niet zonder root-rechten	Hoog	Hoog	Gebruik TCP-connect scan (geen root vereist); documenteren in README
Externe website blokkeert HTTP-verzoek	Middel	Middel	Timeout instellen (5s), foutmelding tonen in UI
HIBP API rate-limiting	Laag	Laag	Verzoeken cachen per sessie; delay inbouwen bij herhaalde calls
Python–Node communicatie faalt	Laag	Hoog	Fallback: Node.js draait zelf de header-check als Python onbereikbaar is

Risico	Kans	Impact	Aanpak
Wachtwoord zichtbaar in logs	Middel	Hoog	Wachtwoord nooit loggen; alleen de eerste 5 hash-chars sturen naar HIBP
Poortscan op verboden host	Middel	Hoog	Disclaimer in UI; blokkering van private IP-ranges (RFC1918) in backend
Tijdgebrek door nmap-complexiteit	Middel	Middel	Nmap als optioneel markeren; basis TCP-check als fallback implementeren

11. Ethische overwegingen

Dit project bevat functionaliteit (poortscannen) die misbruikt kan worden. De volgende maatregelen worden genomen:

- Een duidelijke **disclaimer** in de UI: “Scan alleen websites waarvoor je toestemming hebt.”
- **Blokkering van private IP-ranges** (192.168.x.x, 10.x.x.x, 127.x.x.x) in de backend.
- Het wachtwoord verlaat de browser **nooit in plaintext** – alleen de eerste 5 tekens van de SHA-1 hash gaan naar HIBP.
- De applicatie is bedoeld voor **educatief gebruik**.

12. Vereisten voor oplevering

- ☐ Applicatie start lokaal op met gedocumenteerde installatie-instructies
- ☐ Web Security Scanner toont headers, poorten, kwetsbaarheden en rapport
- ☐ Password Analyzer toont score, entropie, patronen en breach-resultaat
- ☐ HIBP-integratie werkt via k-anonimiteit (wachtwoord wordt niet verstuurd)
- ☐ Code staat op een publieke GitHub-repository
- ☐ README bevat: beschrijving, installatie-instructies en screenshots
- ☐ Reflectieverslag ingeleverd

13. Bronnen & referenties

- MDN Web Docs – HTTP Security Headers
- OWASP Secure Headers Project
- Have I Been Pwned – Passwords API
- zxcvbn – Password Strength Estimator
- python-nmap documentatie
- FastAPI documentatie
- React documentatie
- Vite documentatie
- NIST – Password Guidelines (SP 800-63B)

Plan opgesteld op basis van het POA-aanvraagformulier, mei 2026.